Tartalomjegyzék:

1. Bevezetés ... 1
1.1. Bevezetés ... 1
2. Általunk használt szoftverek ... 2
2.1. Discord: ... 2
2.2. Youtube: ... 2
2.3. GitHub: ... 2
2.4. GitHub Desktop: ... 3
2.5. Gemini: ... 3
2.6. Visual Studio Code: ... 3
2.7. Dia: ... 3
2.8. Cloudflare: ... 4
3. Munkamegosztás ... 5
3.1. Munkamegosztás: ... 5
3.2. Az adatbázis kialakítása: ... 5
3.3. A backend fejlesztése: ... 5
3.4. A frontend fejlesztése: ... 5
4. Adatbázis ... 6
4.1. Adatbázis tervezés ... 6
4.2. Táblák és azok szerepei ... 6
4.2.1. felhasznalo (felhasználó) ... 6
4.2.2. fodrasz (fodrász) ... 6
4.2.3. idopont (időpont) ... 6
4.2.4. munkaido (munkaidő) ... 7
4.2.5. sqlite_sequence ... 7
4.2.6. szolgaltatas (szolgáltatás) ... 7
4.3. Adatbázis felépítése ... 8
5. Backend ... 9
5.1. Bevezetés ... 9
5.2. Mappastruktúra ... 9
5.3. Alkalmazott Backend csomagok ... 10
5.3.1. Express ... 10
5.3.2. CORS ... 10
5.3.3. Dotenv ... 10
5.3.4. Bcrypt ... 10
5.4. Routs ... 10

5.4.1. ugyfelRoutes.js 10
5.4.2. idopontRoutes.js 10
5.4.3. szolgaltatasRoutes.js 10
5.4.4. fodraszRoutes.js 10
5.4.5. munkaidoRoutes.js 11
5.5. Controllers 11
5.5.1. ugyfelController.js 11
5.5.2. idopontController.js 11
5.5.3. szolgaltatasController.js 11
5.5.4. fodraszController.js 11
5.5.5. munkaidoController.js 11
6. Frontend 12
6.1. Bevezetés 12
6.2. Index.html 12
6.3. src mappa 12
6.3.1. app.css: 12
6.3.2. head-foot.css 12
6.3.3. index.css 12
6.4. Pages mappa 13
6.4.1. home.jsx 13
6.4.2. Register.jsx 13
6.4.3. Login.jsx 13
6.4.4. Profile.jsx 13
6.4.5. Naptar.jsx 14
7. Admin felület 14
7.1. Szigorú hozzáférés-szabályozás 14
7.2. Felhasználók kezelése (users fül) 14
7.3. Időpontok felügyelete (appointments fül) 15
7.4. Felhasználói élmény és biztonság 15
8. Tesztelés 16
8.1. Bevezetés 16
8.2. White box tesztelés 16
8.2.1. WBC-01: Adatbázis hitelesítés hibakezelésének tesztelése (db.js) 16
8.2.2. WBC-02: Automatikus karbantartó (Cleanup) szkript lefutatása (index.js) 16
8.2.3. WBC-03: Nem létező végpontok (404 Fallback) tesztelése (index.js) 16
8.2.4. WBC-04: Adatbázis wrapper függvények adatformázása (db.js) 17

8.3. Black box tesztelés 17
8.3.1. BBT-01: Szolgáltatások listájának lekérése (API Teszt - Postman) 17
8.3.2. BBT-02: Bejelentkezés hibás hitelesítő adatokkal (Biztonsági teszt) 17
8.3.3. BBT-04: Meglévő időpont sikeres lemondása (End-to-End teszt) 18
9. Összegzés 19
9.1. Nehézségek 19
9.2. További fejlesztési lehetőségek 19

1. Bevezetés

1.1. Bevezetés

Vizsgaremekünk középpontjában a modern szépségápolás egyik kulcsfontosságú eleme, a szolgáltatások digitalizálása és az ügyfélélmény javítása áll. Célunk, hogy innovatív megoldást kínáljunk a fodrászati időpontfoglalás folyamatára, megkönnyítve ezzel mind a szakemberek, mind a vendégek mindennapjait.

A mai felgyorsult világban az idő az egyik legfontosabb érték, és a hagyományos, telefonos vagy személyes bejelentkezés sokszor nehézkes és időigényes folyamat. Ezen túlmenően a manuális naptárvezetés magasabb hibalehetőséget rejt magában, ami rontja a szolgáltatás hatékonyságát és az üzleti megbízhatóságot.

Vizsgaremekünkkel szeretnénk hozzájárulni a szépségipar modernizációjához, és olyan digitális platformot kínálni, amely lehetővé teszi a 0-24 órás, azonnali időpontfoglalást és a szolgáltatások átlátható kezelését.

Reményeink szerint vizsgaremekünkkel sikerül felhívnunk a figyelmet a digitalizáció előnyeire a szolgáltató szektorban, és motiválni a kisvállalkozásokat a modernebb munkafolyamatok bevezetésére. Hiszünk abban, hogy a megfelelő technológiai háttérrel és eszközökkel minden fodrászszalon képes lehet a növekedésre és egy elégedettebb, lojálisabb vendégkör kialakítására.

2. Általunk használt szoftverek

2.1. Discord:

A Discord volt számunkra a tökéletes platform a kommunikációra. A szerver struktúrája lehetővé tette, hogy átláthatóan elszeparáljuk egymástól az adatbázist, a frontendet és a backendet, ami nagyban megkönnyítette a releváns fájlok megosztását és a fókuszált szakmai kommunikációt.

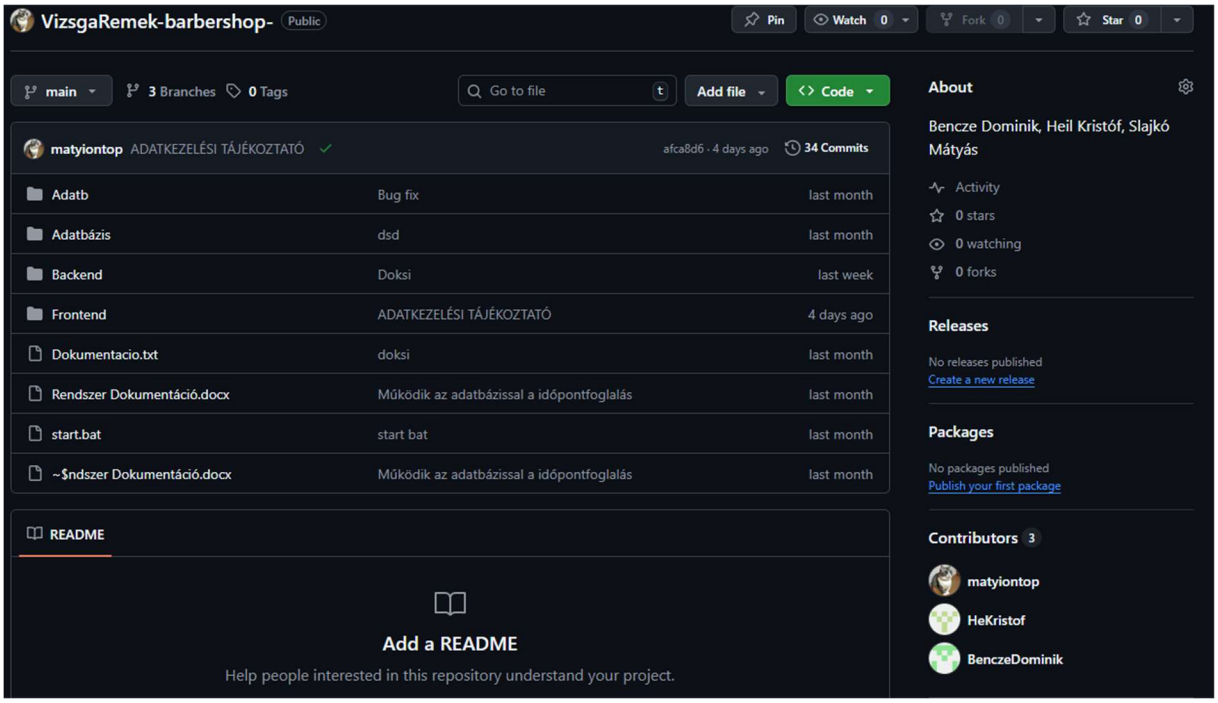
Discord szerver, amelyet a vizsgaremek elkészítése közben használtunk.

2.2. Youtube:

Az ismereteink bővítéséhez és a felmerülő technikai akadályok leküzdéséhez a YouTube szolgált forrásként; az ott található tutorialokat és esettanulmányokat hívtuk segítségül a fejlesztés során.

2.3. GitHub:

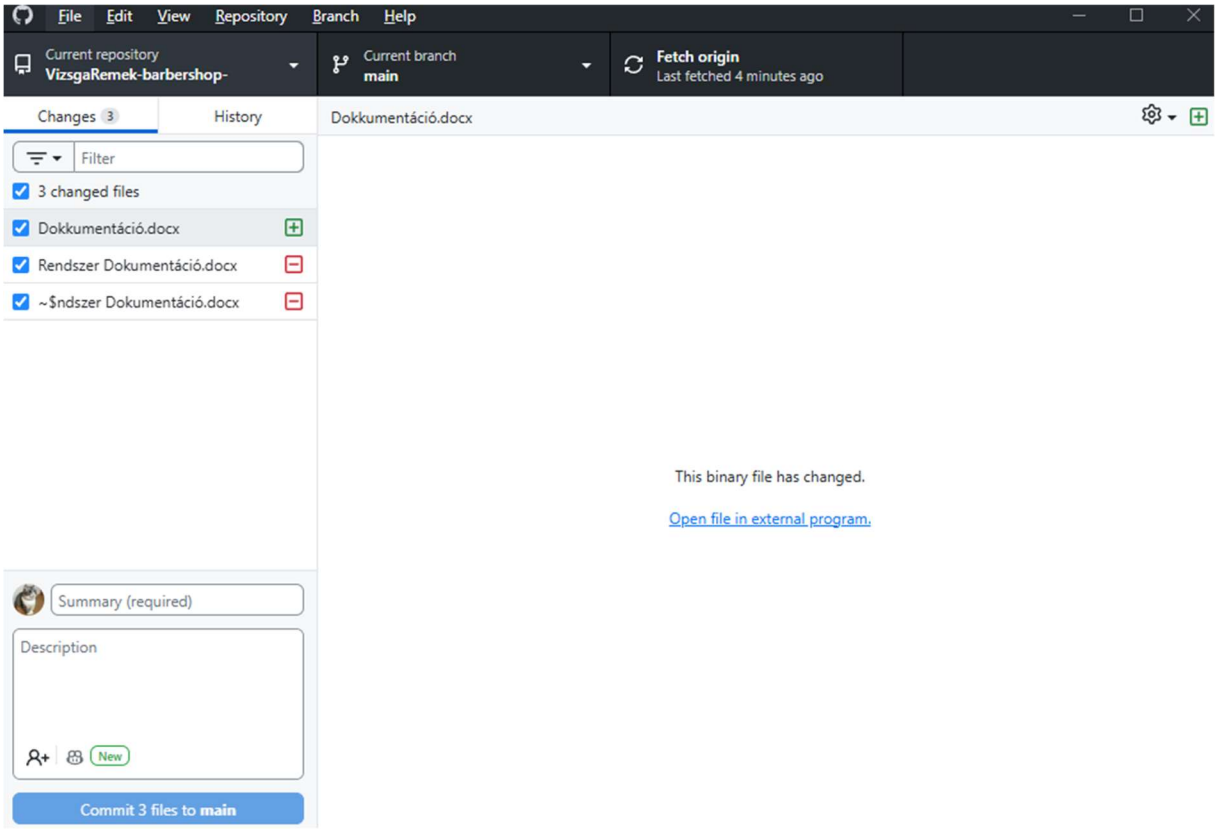
A szoftver fejlesztése során a GitHub felületét alkalmaztuk a forráskód tárolására és a verziókövetésre.



GitHub repositorie.

2.4. GitHub Desktop:

A projekt verziókövetéséhez a GitHub Desktop alkalmazást használtuk. Ez a grafikus felülettel rendelkező eszköz lehetővé tette számunkra a Git funkcióinak egyszerű és átlátható kezelését.



GitHub Desktop

2.5. Gemini:

A munkafolyamatot a Gemini mesterséges intelligencia integrálásával tettük hatékonyabbá. Az AI nemcsak a specifikus programozási problémák elhárításában nyújtott támogatást, hanem az új technológiák elsajátítását és a tanulási folyamatot is jelentősen felgyorsította.

2.6. Visual Studio Code:

A teljes projekt – mind a frontend, mind a backend – kódjának megírásához a Visual Studio Code-ot alkalmaztuk. Ez a környezet tökéletes alapot nyújtott a különböző technológiák egyidejű és átlátható kezeléséhez.

2.7. Dia:

A projekt adatbázisának megtervezését a Dia program segítségével végeztük el. Ebben a szoftverben rajzoltuk meg a rendszer EK (Egyed-Kapcsolat) diagramját, ami elengedhetetlen volt az adattárolási logika és a táblák közötti összefüggések pontos definiálásához.

2.8. Cloudflare:

A projekt felhőalapú adatbázisának létrehozásához és üzemeltetéséhez a Cloudflare szolgáltatását alkalmaztuk. A rendszerünk adatbázisa jelenleg is ezen a modern, felhőalapú infrastruktúrán fut, amely garantálja a folyamatos elérhetőséget, a skálázhatóságot és a gyors válaszidőket.

3. Munkamegosztás

3.1. Munkamegosztás:

A feladatok elosztásánál elsődleges szempontnak tekintettük, hogy minden csapattag a saját erősségeinek és szaktudásának megfelelő területen dolgozzon, ezzel is maximalizálva a projekt hatékonyságát.

3.2. Az adatbázis kialakítása:

A projekt adattárolási modelljének megtervezése és implementálása Bencze Dominik feladata volt. Fő célkitűzése egy olyan normalizált és optimalizált adatbázis létrehozása volt, amely felesleges ismétlődések (redundancia) nélkül, a lehető legkisebb helyigénnyel működik. Ennek a precíz tervezésnek köszönhetően a szerver és a kliensoldal közötti adatkommunikáció kifejezetten gyorssá és stabillá vált.

3.3. A backend fejlesztése:

A rendszer motorjának leprogramozását Slajkó Mátyás hajtotta végre. Munkája során a „Clean Code" (tiszta kód) paradigmát követte, így egy átlátható, jól skálázható kódbázist épített fel. A szerveroldal logikáját egy tudatosan megszervezett könyvtárstruktúrával tette rendszerezetté, ami nemcsak az alkalmazás válaszidejét optimalizálta, hanem a későbbi karbantartást és továbbfejlesztést is jelentősen megkönnyíti.

3.4. A frontend fejlesztése:

A publikus weboldal és az adminisztrációs panel vizuális megjelenéséért, valamint kliensoldali programozásáért Heil Kristóf felelt. A backend fejlesztéshez igazodva ő is a jól strukturált, moduláris kódolási elveket alkalmazta. Kiemelt figyelmet fordított a projekt arculatára: egy letisztult, modern és rendkívül felhasználóbarát felületet alkotott meg, amely tökéletes ergonómiai összhangban áll a beépített funkciókkal.

4. Adatbázis

4.1. Adatbázis tervezés

Az alkalmazás működéséhez szükséges adatokat SQLite alapú adatbázisban tároltam.

Az adatbázis létrehozása és kezelése Cloudflare környezetben történt, amely lehetővé teszi a gyors és megbízható adatkezelést.

Az adatbázis több egymással kapcsolatban álló táblából épül fel, amelyek a rendszer különböző elemeit reprezentálják (pl. felhasználók, szolgáltatások, időpontok). A táblák között idegen kulcsok biztosítják az adatok közötti kapcsolatokat és az adatintegritást.

A tervezés során törekedtem arra, hogy az adatok strukturáltan, redundancia nélkül kerüljenek tárolásra, valamint, hogy a rendszer könnyen bővíthető és átlátható maradjon.

4.2. Táblák és azok szerepei

4.2.1. felhasznalo (felhasználó)

A felhasználókat tárolja el.

- **felhasznalo_id** (INTEGER, elsődleges kulcs)
- **nev** (TEXT, NOT NULL):
- **email** (TEXT, NOT NULL, UNIQUE):
- **jelszo** (TEXT, NOT NULL):
- **engedely** (INTEGER, NOT NULL, DEFAULT 1):
- **szerep** (TEXT, NOT NULL, DEFAULT 'U')

Kapcsolatok:

- Kapcsolódik az idopont táblához.

4.2.2. fodrasz (fodrász)

Ebben a tárolóban tároljuk el a fodrászt.

- **fodrasz_id** (INTEGER, elsődleges kulcs): Egyedi azonosító.
- **nev** (TEXT, NOT NULL): A fodrász neve.

Kapcsolatok:

- Kapcsolódik az idopont és a munkaido táblákhoz.

4.2.3. idopont (időpont)

Az időpontot tárolja le.

- **idopont_id** (INTEGER, elsődleges kulcs): Egyedi azonosító.
- **felhasznalo_id** (INTEGER, idegen kulcs a felhasznalo.felhasznalo_id-ra): Felhasználó azonosítója.
- **fodrasz_id** (INTEGER, idegen kulcs a fodrasz.fodrasz_id-ra): Fodrász azonosítója.
- **szolgaltatas_id** (INTEGER, idegen kulcs a szolgaltatas.szolgaltatas_id-ra): Szolgáltatás azonosítója.
- **idopont_datuma** (TEXT, NOT NULL): Időpont dátuma.
- **kezdesi_ido** (TEXT, NOT NULL): A kezdés időpontja.

- **allapot** (TEXT, DEFAULT 'foglalva'): Időpont állapota.

Kapcsolatok:

- A felhasznalo táblához kapcsolódik a felhasznalo_id mezővel.
- A fodrasz táblához kapcsolódik a fodrasz_id mezővel.
- A szolgaltatas táblához kapcsolódik a szolgaltatas_id mezővel.

4.2.4. munkaido (munkaidő)

A fodrász munkaidejét tárolja.

- **munkaido_id** (INTEGER, elsődleges kulcs): Egyedi azonosító.
- **fodrasz_id** (INTEGER, idegen kulcs a fodrasz.fodrasz_id-ra): Fodrász azonosítója.
- **het_napja** (TEXT): A hét napjai.
- **nyitas** (TEXT, NOT NULL): A munkaidő kezdete.
- **zaras** (TEXT, NOT NULL): A munkaidő vége.
- **ebed_kezdete** (TEXT): Az ebéd kezdete.
- **ebed_vege** (TEXT): A ebéd vége.

Kapcsolatok:

- A fodrasz táblához kapcsolódik a fodrasz_id mezővel.

4.2.5. sqlite_sequence

Az SQLite által automatikusan generált rendszer tábla, amely az AUTOINCREMENT kulccsal rendelkező táblákhoz tartozó számláló aktuális értékét tárolja.

- **rowid** (INTEGER, elsődleges kulcs): Egyedi azonosító.
- **name** (TEXT): Azon táblák neve, amelyek AUTOINCREMENT mezőt tartalmaznak.
- **seq** (INTEGER): Az adott tábla AUTOINCREMENT mezőjének aktuális legnagyobb értéke.

4.2.6. szolgaltatas (szolgáltatás)

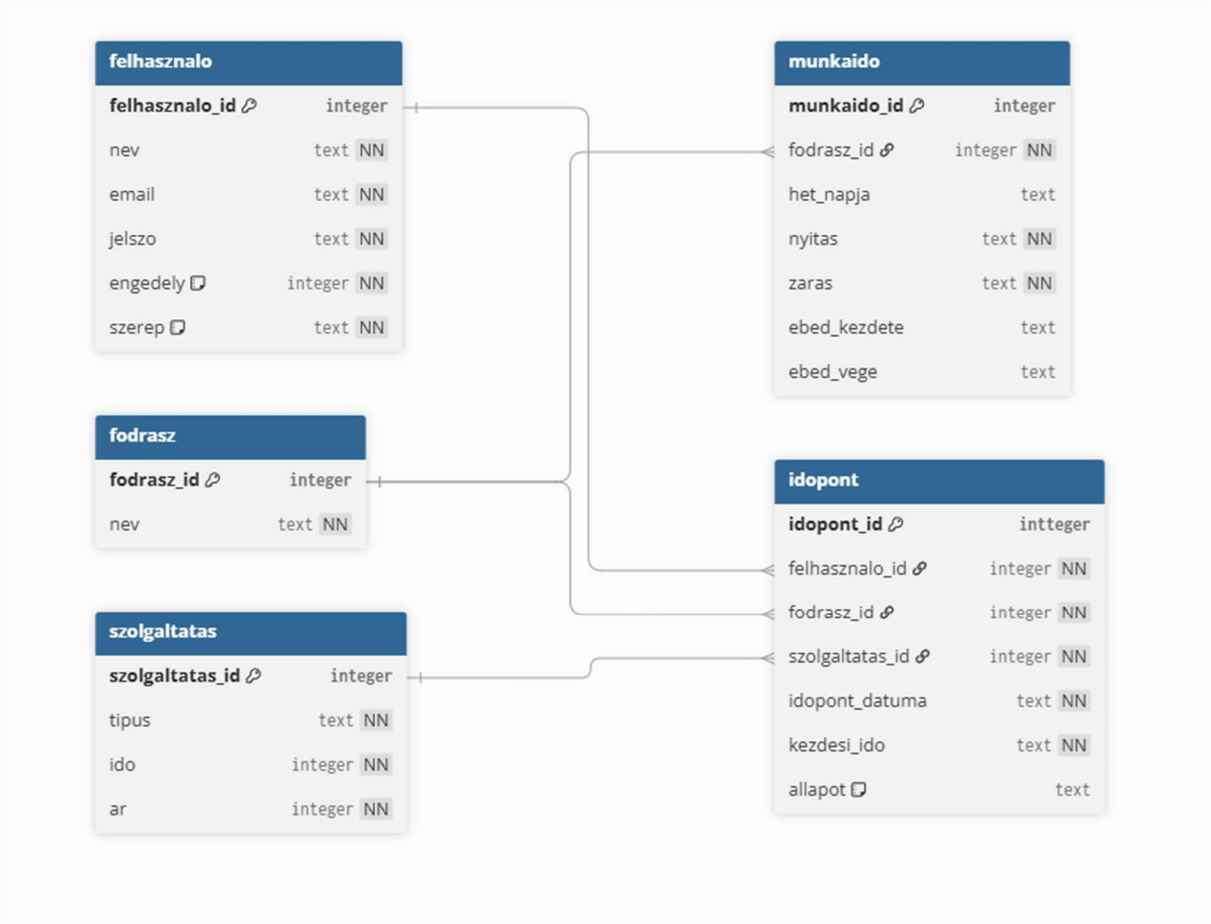
A szolgáltatásokat tárolja el.

- **szolgaltatas_id** (INTEGER, elsődleges kulcs): Egyedi azonosító.
- **tipus** (TEXT, NOT NULL): A szolgáltatás tipusa.
- **ido** (INTEGER, NOT NULL): A szolgáltatás időtartalma 20 perces egységekben megadva (pl. 1 = 20 perc).
- **ar** (INTEGER, NOT NULL): A szolgáltatás ára.

Kapcsolatok:

- Kapcsolódik az idopont táblához.

4.3. Adatbázis felépítése



kép: https://dbdiagram.io/d

5. Backend

5.1. Bevezetés

Az adatkezelést és a komplex műveletek végrehajtását, az adatbiztonságért, valamint az adatbázissal történő kommunikációért a szerveroldali (backend) egysége felel. Az alkalmazás Node.js futtatókörnyezetben és a Express.js keretrendszer felhasználásával készült. Ez biztosítja a gyors, robusztus és skálázható RESTful API végpontok kialakítását, amelyeken keresztül a kliensoldali alkalmazás (frontend) JSON formátumú adatcserét folytat a szerverrel.

A backend felépítése moduláris, és az MVC (Model-View-Controller) tervezési minta alapelveit követi. A rendszer két legfőbb strukturális pillérét az útvonalak (Routes) és a vezérlők (Controllers) alkotják.

5.2. Mappastruktúra

- **index.js**: A szerveralkalmazás fő belépési pontja. Itt történik az Express keretrendszer inicializálása, a globális beállítások (middleware-ek) megadása, valamint az útvonalak (routes) becsatolása.
- **db.js:** Az adatbázis-kapcsolatért felelős modul. Ez a fájl kezeli a hitelesítést és a lekérdezések futtatását a felhő alapú Cloudflare D1 adatbázis felé.
- **routes/ mappa:** Az API végpontok (endpointok) definícióit tartalmazza. Az itteni fájlok (pl. ugyfelRoutes.js) fogadják a HTTP kéréseket, és tovább irányítják azokat a megfelelő vezérlőknek.
- **controllers/ mappa:** A rendszer üzleti logikáját (pl. idopontController.js) megvalósító fájlok helye. Itt történik az adatok validálása, a számítások elvégzése és az adatbázis-műveletek végrehajtása.
- **node_modules/ mappa:** Itt tároljuk a projekt működéséhez szükséges, automatikusan letöltött külső könyvtárakat.

controllers	2026. 02. 05. 10:59	Fájlmappa	
node_modules	2026. 01. 26. 10:20	Fájlmappa	
routes	2026. 02. 05. 10:59	Fájlmappa	
.env	2026. 01. 26. 11:06	ENV fájl	1 KB
api.xlsx	2026. 01. 15. 10:37	Microsoft Excel-m...	9 KB
db.js	2026. 01. 26. 11:15	JavaScript forrásfájl	3 KB
index.js	2026. 03. 02. 9:09	JavaScript forrásfájl	2 KB
kepsegit.png	2026. 02. 05. 10:17	PNG fájl	426 KB
package.json	2026. 01. 26. 10:20	JSON forrásfájl	1 KB
package-lock.json	2026. 01. 26. 10:20	JSON forrásfájl	44 KB
packages.txt	2026. 01. 26. 11:40	Szöveges dokume...	1 KB
RUN.bat	2026. 02. 10. 10:27	Windows kötegfájl	1 KB

Backend mappa

5.3. Alkalmazott Backend csomagok

5.3.1. Express

A webszerver és az API végpontok (útvonalak) létrehozására, valamint a beérkező HTTP kérések feldolgozására szolgáló alapkeretrendszer.

5.3.2. CORS

Engedélyezi a kereszt-eredetű kéréseket, ezáltal biztosítja, hogy a külön porton futó React frontend kommunikálhasson a backend szerverrel.

5.3.3. Dotenv

Az érzékeny adatok (pl. Cloudflare adatbázis azonosítók, API tokenek) biztonságos beolvasására használjuk egy külső. env fájlból.

5.3.4. Bcrypt

A felhasználói jelszavak biztonságos, egyirányú titkosításáért (hash-eléséért) felel a regisztráció és bejelentkezés során.

5.4. Routs

5.4.1. ugyfelRoutes.js

A felhasználói fiókok hitelesítéséért és a jogosultságkezelésért felel.

- **POST /register, POST /login:** Regisztráció és bejelentkezés.
- **GET /:** Az összes regisztrált felhasználó adatainak lekérése (Admin funkció).
- **PUT /:id:** Meglévő felhasználó adatainak, jelszavának vagy szerepkörének módosítása.
- **DELETE /:id:** Felhasználói fiók törlése.

5.4.2. idopontRoutes.js

Az időpontfoglalási rendszer központi útvonalai.

- **GET /:** Az összes foglalás lekérdezése (Naptár kirajzolásához).
- **GET /user/:id:** Egy adott ügyfél saját foglalásainak lekérése.
- **POST /:** Új foglalási igény rögzítése a rendszerben.
- **POST /cancel/:id:** Időpont lemondása a felhasználó által (a 24 órás szabály ellenőrzésével).
- **DELETE /:id:** Időpont azonnali törlése (Admin funkció).
- **DELETE /cleanup:** A lejárt, múltbeli időpontok automatikus adatbázis-tisztítását (karbantartását) végző végpont.

5.4.3. szolgaltatasRoutes.js

A fodrászat szolgáltatásainak és árainak lekérdezése (GET).

- **GET /:** A fodrászat által nyújtott szolgáltatások, azok árainak és időtartamainak lekérdezése.

5.4.4. fodraszRoutes.js

Az elérhető fodrászok listájának lekérése (GET).

- **GET /:** Az elérhető fodrászok listájának betöltése az időpontfoglaló felülethez.

5.4.5. munkaidoRoutes.js

A nyitvatartási és beosztási adatok szolgáltatása (GET)

- **GET /:** A szalon általános nyitvatartási rendjének és a fodrászok beosztásának lekérdezése.

5.5. Controllers

5.5.1. ugyfelController.js

A felhasználók és a hitelesítés (autentikáció) kezelését végzi.

- **Regisztráció és Bejelentkezés:** Fogadja a felhasználói adatokat, a bcrypt csomag segítségével titkosítja a jelszavakat, és ellenőrzi a belépési kísérleteket.
- **Adminisztráció:** Biztosítja a regisztrált ügyfelek listázását, adataiknak (név, email, jogosultsági kör) módosítását, illetve a fiókok biztonságos törlését az adatbázisból.

5.5.2. idopontController.js

A foglalási rendszer „agya”, ez a legösszetettebb modul.

- **Foglalás és Validáció:** Új időpont rögzítésekor ellenőrzi az adatok helyességét, összekapcsolja az ügyfelet, a fodrászt és a szolgáltatást, valamint megakadályozza a naptári ütközéseket (dupla foglalás).
- **Lekérdezések:** Kiszolgálja a naptár nézetet az összes foglalás átadásával, illetve biztosítja, hogy a bejelentkezett felhasználók láthassák a saját jövőbeli időpontjaikat.
- **Lemondás és Tisztítás:** Kezeli a felhasználó általi lemondásokat, valamint tartalmazza a lejárt (múltbeli) időpontok automatikus törlését végző karbantartó logikát.

5.5.3. szolgaltatasController.js

Lekérdezi és strukturált formában átadja a frontend számára a fodrászat által kínált szolgáltatásokat, a hozzájuk tartozó aktuális árakkal és a becsült időtartamokkal (percben) együtt.

5.5.4. fodraszController.js

Az adatbázisból lekéri a szalonban dolgozó fodrászok nevét és azonosítóit, amelyek elengedhetetlenek az időpontfoglaló felület legördülő menüinek feltöltéséhez.

5.5.5. munkaidoController.js

Felel a szalon nyitvatartási rendjének, illetve az egyes fodrászokhoz tartozó műszakkezdési és befejezési időpontoknak a lekérdezéséért, ezáltal biztosítva, hogy a naptár csak a valós munkaidőben engedjen foglalást.

6. Frontend

6.1. Bevezetés

Minden modern webes alkalmazás életútja egyetlen központi pontból indul ki, amely keretet ad a teljes felhasználói élménynek. Ez az **index.html** fájl nem csupán egy kódhalmaz, hanem a projektünk digitális alapköve: itt találkozik a böngésző az alkalmazásunkkal, itt dőlnek el az alapvető megjelenítési szabályok, és innen indul útjára az a komplex logika, amelyet a háttérben felépítettünk. Ez a fájl biztosítja a stabilitást és a kapcsolódási pontot a nyers kód és a látható felület között.

6.2. Index.html

- **Alapbeállítások:** Itt rögzítjük a magyar nyelv használatát, a helyes karakterkijelzést és a mobilbarát megjelenítést. (pl.: lang="hu", title, charset,)
- **Befogadó szerkezet:** Tartalmazza a root azonosítójú elemet, amely az alkalmazás teljes vizuális tartalmának technikai "gazdateste".
- **Indítás:** Összeköti a statikus HTML oldalt a modern JavaScript logikával (main.jsx), így keltve életre a programot.

6.3. src mappa

Az src mappában számos file található, css és jsx filok

6.3.1. app.css:

- **Elrendezés**: Meghatározza a tartalom maximális szélességét (1280px) és gondoskodik a képernyő közepére igazításról.
- **Interaktivitás**: Beállítja a logók feletti „világító" (glow) effektust az egér használatakor.
- **Animáció**: Tartalmazza a logók folyamatos forgását szabályozó logikát.
- **Felhasználói élmény:** Automatikusan leállítja a mozgásokat, ha a látogató rendszerszinten tiltja az animációkat (pl. akadálymentesség miatt).

6.3.2. head-foot.css

- **Fejléc (Header):** Beállít egy látványos háttérképet, amely a képernyő magasságának 35%-át foglalja el. Itt helyezi el a főcímet a bal felső sarokba, a navigációs gombokat pedig a jobb alsó sarokba.
- **Lábléc (Footer):** Egy sötét tónusú, kontrasztos lezárást ad az oldalnak, ahol a tartalmi elemeket (pl. elérhetőségek, linkek) rugalmasan, egymás mellé rendezve jeleníti meg.
- **Intelligens elrendezés (Sticky Footer):** A .app-container és a main segítségével gondoskodik róla, hogy a lábléc akkor is az oldal alján maradjon, ha kevés a tartalom, így elkerülhető a csúnya "felcsúszó" footer.

6.3.3. index.css

- **Globális alapok:** Beállítja a betűtípusokat, a szövegméreteket és az alapértelmezett háttérszínt. Gondoskodik a modern, simított betűmegjelenítésről.
- **Adaptív színvilág (Dark/Light mode):** Tartalmaz egy intelligens színkezelést, amely igazodik a felhasználó operációs rendszeréhez. Alapértelmezetten sötét témát használ, de automatikusan világosra vált, ha a látogató azt preferálja.
- **Központi UI elemek:** Egységes stílust ad a gomboknak és linkeknek, beleértve az interaktív állapotokat (pl.: ha fölé viszed az egeret vagy rákattintasz).

- **Aloldalak specifikus stílusai:**
    - **Home:** A nyitóoldal rugalmas (Flexbox) elrendezéséért és a képek megjelenítéséért felel.
    - **Auth (Bejelentkezés/Regisztráció):** Keretet és formát ad az űrlapoknak, hogy azok mobilról és asztali gépről is kényelmesen használhatóak legyenek.

Admin felület: Itt található a legösszetettebb rész; szabályozza a felhasználói listák, keresőmezők és szerkesztőablakok kinézetét, biztosítva a professzionális kezelőfelületet.

6.4. Pages mappa

6.4.1. home.jsx

- **Kettős felépítés:** A komponens két fő információs blokkra oszlik: egy személyes bemutatkozásra (Rólam) és a szolgáltatások ismertetésére (Hajvágás).
- **Vizuális tartalom:** Képekkel (fodrasz.png, hajvagas.png) teszi szemléletessé és barátságossá a szolgáltatást, segítve a bizalomépítést.
- **Rugalmas elrendezés:** A kepekegymasmellett osztály és az inline Flexbox formázás gondoskodik róla, hogy a szöveg és a kép rendezetten, egymás mellett (vagy mobilon egymás alatt) jelenjen meg.
- **Letisztult dizájn:** A fix szélességek (maxWidth) és a térközök (gap) használatával a tartalom szellős, jól olvasható és professzionális hatást kelt.

6.4.2. Register.jsx

- **Adatkezelés:** A useState hook segítségével valós időben követi a felhasználó által megadott nevet, e-mail címet és jelszót.
- **API kommunikáció:** A regisztrációs gomb megnyomásakor egy aszinkron POST kérést indít a háttérszerver felé (/api/ugyfelek/register), továbbítva a megadott adatokat JSON formátumban.
- **Felhasználói visszajelzés:** Az RSuite könyvtár useToaster szolgáltatását használja a folyamat eredményének jelzésére:
    - **Siker esetén:** Pozitív üzenetet jelenít meg, majd automatikusan átirányítja a felhasználót a bejelentkezési oldalra.
    - **Hiba esetén:** Részletes hibaüzenetet közöl (pl. már létező e-mail cím vagy szererelérési hiba).
- **Biztonság és validáció:** Az űrlap alapvető HTML5 validációt használ (kötelező mezők, e-mail formátum), megelőzve az üres vagy hibás adatok beküldését.

6.4.3. Login.jsx

- **Hitelesítés:** Egy aszinkron POST kérést küld a szervernek, amely ellenőrzi az e-mail cím és a jelszó párosát.
- **Munkamenet kezelése:** Sikeres belépés esetén a felhasználói adatokat a localStorage-ba menti, így az alkalmazás a böngésző bezárása után is emlékszik a bejelentkezett felhasználóra.
- **Dinamikus visszajelzés:** Az RSuite értesítési rendszerével (Toaster) azonnali visszajelzést ad a felhasználónak a belépés sikeréről vagy az esetleges hibákról.
- **Állapotfrissítés:** Sikeres bejelentkezés után az oldal kényszerített újratöltést végez, hogy a navigációs menü (pl. a kijelentkezés gomb vagy az admin funkciók) azonnal frissüljön.

6.4.4. Profile.jsx

- **Jogosultságkezelés:** Belépéskor ellenőrzi, hogy a felhasználó be van-e jelentkezve. Ha nem talál érvényes munkamenetet, automatikusan a bejelentkezési oldalra irányítja a látogatót.
- **Időpontok megjelenítése:** Egy átlátható táblázatba rendezve listázza a felhasználó összes aktív foglalását (dátum, időpont, szolgáltatás típusa, ár és a választott fodrász).

- **Üzleti logika (24 órás szabály):** A rendszer automatikusan kalkulálja, hogy az adott időpont lemondható-e még. A gomb csak akkor aktív, ha a kezdésig még legalább 24 óra hátra van, ezzel segítve a szalon működésének tervezhetőségét.
- **Biztonságos lemondás:** Megakadályozza a véletlen törlést egy megerősítő felugró ablak (Modal) használatával. A törlési folyamat végén a lista azonnal frissül, és a felhasználó visszajelzést kap a művelet sikerességéről.

6.4.5. Naptar.jsx

Intelligens időpontkezelés

- **20 perces egységek:** A rendszer nem fix órákkal, hanem 20 perces blokkokkal számol. A szolgáltatások (pl. hajvágás vs. teljes festés) hossza alapján a komponens kiszámolja, hány egymást követő szabad blokkra van szükség a foglaláshoz.
- **Dinamikus naptárkép:** A naptár cellái színes pöttyökkel (Badge) jelzik a nap telítettségét:
  - **Zöld:** Bőven van hely.
  - **Sárga:** A nap több mint fele már foglalt.
  - **Piros:** Nincs elegendő összefüggő szabad hely a választott szolgáltatáshoz.

Automatikus szűrés és validáció

- **Munkaidő és ebédidő:** A naptár figyelembe veszi a fodrász nyitvatartását és az előre rögzített ebédidőt, ezeket a sávokat automatikusan zárolja.
- **Ütközésfigyelés:** A foglalás gomb csak akkor jelenik meg, ha a kiválasztott időpont és a szolgáltatás hossza nem ütközik egy már meglévő vendég foglalásával vagy az ebédidővel.
- **Múltbéli tiltás:** A rendszer megakadályozza, hogy a felhasználók visszamenőlegesen vagy lezárt napokra foglaljanak.

Felhasználói folyamat

- **Adatbetöltés:** Induláskor lekéri a szolgáltatások listáját, az árakat, a fodrászok elérhetőségét és a meglévő foglalásokat.
- **Foglalás menete:** Ellenőrzi a bejelentkezési státuszt, összegyűjti a szükséges azonosítókat (felhasználó, fodrász, szolgáltatás), majd egyetlen POST kéréssel véglegesíti az időpontot.

7. Admin felület

7.1. Szigorú hozzáférés-szabályozás

A biztonság érdekében a komponens betöltésekor azonnal ellenőrzi a felhasználó státuszát. Ha valaki bejelentkezés nélkül vagy nem adminisztrátori jogosultsággal próbálja elérni ezt a felületet, a rendszer automatikusan visszairányítja a főoldalra.

7.2. Felhasználók kezelése (users fül)

- Keresés és listázás: Az adminisztrátor név alapján szűrheti a regisztrált ügyfeleket.
- Szerkesztés: Lehetőséget ad az adatok (név, e-mail) módosítására, új jelszó beállítására, vagy a felhasználó teljes letiltására/engedélyezésére.
- Törlés: Véglegesen eltávolítható egy felhasználó a rendszerből (biztonsági megerősítés után).

7.3. Időpontok felügyelete (appointments fül)

- **Globális áttekintés:** Egy táblázatos nézetben megjeleníti az összes jövőbeli és múltbéli foglalást.
- **Intelligens szűrés:** Gyors keresést tesz lehetővé ügyfélnév, dátum vagy konkrét óra/perc alapján.
- **Rendkívüli törlés:** Az adminisztrátornak joga van bármelyik időpontot törölni (például váratlan betegség vagy technikai okok miatt), amit egy megerősítő ablak véd a véletlen kattintásoktól.

7.4. Felhasználói élmény és biztonság

- **Megerősítő ablakok (Modals):** A kritikus műveletek (törlések) nem futnak le azonnal; egy felugró ablak kéri az adminisztrátor jóváhagyását, megelőzve az adatvesztést.

Valós idejű visszajelzés: Minden sikeres mentésről vagy törlésről látványos értesítést (Toast) küld a rendszer.

8. Tesztelés

8.1. Bevezetés

A szoftverfejlesztési és minőségbiztosítási folyamat elengedhetetlen része a tesztelés, amely garantálja a rendszer megbízható, biztonságos és az elvárásoknak megfelelő működését. A projekt validálása során két átfogó tesztelési stratégiát alkalmaztunk: a Black box és a White box tesztelést.

Black box tesztelés: Ennek során a rendszer belső kódolásának és logikájának ismerete nélkül, a szoftvert végfelhasználói szemmel vizsgáltuk. Ide tartozott a felhasználói felület (Frontend) manuális tesztelése a böngészőben (pl. gombok működése, űrlapok kitöltése, navigáció), valamint az API végpontok működésének vizsgálata. A szerveroldali (Backend) hálózati útvonalak teszteléséhez a Postman nevű szoftvert alkalmaztuk. Ennek segítségével szimuláltuk a kliensoldali HTTP kéréseket (GET, POST, PUT, DELETE), bemeneti adatokat küldtünk a szervernek, és ellenőriztük, hogy a rendszer a megfelelő állapotkódokkal (pl. 200 OK, 201 Created) és JSON adatokkal tér-e vissza.

White box tesztelés: Ezzel a módszerrel az alkalmazás belső struktúráját, algoritmusait és adatbázis-műveleteit vizsgáltuk. A tesztelés kiterjedt a forráskód helyes lefutására, a vezérlőkben (Controllers) található logikai elágazások ellenőrzésére, a hitelesítési folyamatokra, valamint a hibakezelési mechanizmusok (pl. try-catch blokkok, adatbázis csatlakozási hibák) és az automatizált folyamatok (pl. időpontok napi karbantartása) strukturális validálására.

8.2. White box tesztelés

8.2.1. WBC-01: Adatbázis hitelesítés hibakezelésének tesztelése (db.js)

- **A teszt célja:** Annak ellenőrzése, hogy a rendszer megfelelően lekezeli-e, ha a felhő adatbázissal (Cloudflare D1) való kommunikáció során érvénytelen az API kulcs.
- **Tesztlépés / Bemenet:** A fejlesztői környezetben a .env fájlban szándékosan megváltoztattuk a CF_API_TOKEN értékét egy érvénytelen jelszóra, majd indítottunk egy adatbázis lekérdezést.
- **Elvárt eredmény:** A szerver nem omolhat össze (nem dobhat Unhandled Promise Rejection hibát). Az executeD1 függvény catch ágának kell aktiválódnia, és a konzolon egy strukturált "Cloudflare D1 API Hiba" üzenetnek kell megjelennie a konkrét hibaok megjelölésével.
- **Tényleges eredmény:** Megfelelt. A hiba elkapásra került, a futás nem állt le, a kliens 500-as hibaüzenetet kapott.

8.2.2. WBC-02: Automatikus karbantartó (Cleanup) szkript lefutatása (index.js)

- **A teszt célja:** A szerver indulásakor lefutó, múltbeli időpontokat törlő aszinkron funkció ellenőrzése.
- **Tesztlépés / Bemenet:** A szerver újraindítása (npm start). A kód alapján az index.js fájl alján az idopontController.performCleanup() függvény meghívásra kerül.
- **Elvárt eredmény:** A függvénynek (Promise) sikeresen le kell futnia anélkül, hogy megakasztaná az Express szerver elindulását (amely a PORT-on figyel). Sikeres lefutás esetén a konzolon a "Karbantartás: Régi időpontok sikeresen törölve." üzenetnek kell megjelennie.
- **Tényleges eredmény:** Megfelelt. A szerver elindult, az aszinkron tisztítás a háttérben sikeresen végbement.

8.2.3. WBC-03: Nem létező végpontok (404 Fallback) tesztelése (index.js)

- **A teszt célja:** Az alapértelmezett, mindent elkapó "catch-all" middleware működésének validálása.

- **Tesztlépés / Bemenet:** Kérés küldése egy kódszinten nem definiált API végpontra (pl. GET /api/nemletezo-oldal).
- **Elvárt eredmény:** Mivel egyetlen router sem illeszkedik az útvonalra, a kérésnek el kell érnie az index.js végén található 404-es middleware-t. A válasz egy 404 Not Found HTTP státuszkód kell legyen, pontosan a következő JSON tartalommal: { "error": "Az oldal nem található (404)" }.
- **Tényleges eredmény:** Megfelelt.

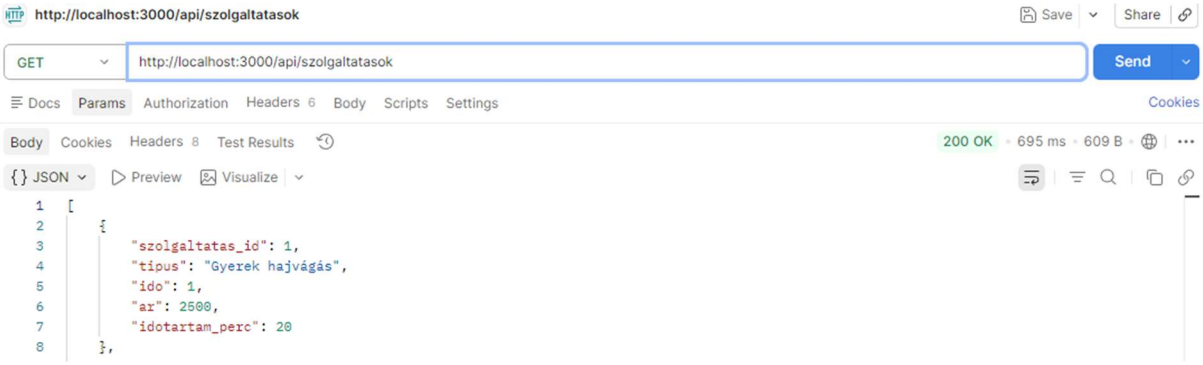
8.2.4. WBC-04: Adatbázis wrapper függvények adatformázása (db.js)

- **A teszt célja:** A D1 adatbázis API-jából érkező komplex JSON válasz megfelelő "kicsomagolásának" tesztelése a dbInterface.get metódusban.
- **Tesztlépés / Bemenet:** Egy olyan SQL lekérdezés futtatása a dbInterface.get() metóduson keresztül, amely garantáltan egyetlen sort ad vissza (pl. egy adott id-jű ügyfél lekérése).
- **Elvárt eredmény:** A metódus nem a Cloudflare által visszaadott teljes tömböt (data.result.results), hanem szigorúan csak annak első elemét (result.results[0]) adja vissza a vezérlőnek. Ha nincs találat, undefined értékkel tér vissza.
- **Tényleges eredmény:** Megfelelt. Az adatszerkezet validációja sikeres volt, a controllerek pontos adatot kapnak.

8.3. Black box tesztelés

8.3.1. BBT-01: Szolgáltatások listájának lekérése (API Teszt - Postman)

- **A teszt célja:** Annak vizsgálata, hogy a backend API megfelelően strukturált adatokat szolgáltat-e a frontend számára külső kérés esetén.
- **Tesztlépés / Bemenet:** HTTP GET kérés szimulálása Postman szoftver segítségével a http://localhost:3000/api/szolgaltatasok végpontra.
- **Elvárt eredmény:** A szerver HTTP 200 OK státuszkóddal válaszol. A válasz törzse (Response Body) egy érvényes JSON tömb, amely pontosan tartalmazza az adatbázisban lévő szolgáltatásokat, azok árait és időtartamát.
- **Tényleges eredmény:** Megfelelt. Az adatok betöltése gyors és a formátum megfelelő.



Postman program amit a teszteléshez használtunk.

8.3.2. BBT-02: Bejelentkezés hibás hitelesítő adatokkal (Biztonsági teszt)

- **A teszt célja:** A rendszer reakciójának ellenőrzése jogosulatlan belépési kísérlet esetén.
- **Tesztlépés / Bemenet:** A bejelentkezési (Login) űrlapon egy létező, regisztrált email cím megadása, de szándékosan hibás jelszó begépelése, majd a kérés elküldése.

- **Elvárt eredmény:** A szerver elutasítja a belépést egy HTTP 401 (Unauthorized) hibaüzenettel. A felhasználói felületen a rendszer nem omlik össze, hanem egy barátságos hibaüzenetet jelenít meg ("Hibás email cím vagy jelszó!").
- **Tényleges eredmény:** Megfelelt. A biztonsági szűrés blokkolta a belépést.

8.3.3. BBT-04: Meglévő időpont sikeres lemondása (End-to-End teszt)

- **A teszt célja:** Egy teljes felhasználói folyamat (lemondás) elejétől a végéig történő tesztelése az UI-n keresztül.
- **Tesztlépés / Bemenet:** A bejelentkezett felhasználó a "Saját időpontjaim" (vagy Naptár) felületen rákattint a "Lemondás" gombra egy jövőbeli foglalása mellett.
- **Elvárt eredmény:** A kattintás után a rendszer kér egy megerősítést (hogy elkerülje a véletlen törlést). Jóváhagyás után a foglalás azonnal eltűnik a képernyőről, a felület frissül. A háttérben egy sikeres DELETE kérés megy végbe a /api/idopontok/cancel/:id végponton (HTTP 200 státusszal).
- **Tényleges eredmény:** Megfelelt. A folyamat zökkenőmentes és a felület is szinkronban marad az adatbázissal

9. Összegzés

9.1. Nehézségek

A weboldal és a mögötte álló rendszer elkészítése során rengeteg új technológiával és koncepcióval találkoztunk. Ez az időszak összességében hatalmas fejlődési lehetőség volt számunkra, de természetesen számos kihívást is tartogatott, hiszen sok olyan dolgot kellett alkalmaznunk, amiket korábban még nem, vagy csak érintőlegesen használtunk.

A legnagyobb feladatot talán a projekt különböző részeinek összehangolása jelentette. Kezdetben komoly fejtörést okozott a felhasználói felület (kliens) és a szerver közötti biztonságos kommunikáció kialakítása, valamint az adatbázis megfelelő integrálása. A modern, felhőalapú rendszerek használata és a hálózaton keresztül érkező adatok feldolgozása teljesen új gondolkodásmódot igényelt, amihez rengeteg tesztelésre és hibakeresésre volt szükség.

Emellett a bonyolultabb funkciók – mint például maga az időpontfoglalási logika, az időpontok szűrése és a naptár kezelése – is próbára tettek minket. Meg kellett értenünk a háttérben futó, időbe telő (aszinkron) folyamatok működését, hogy a weboldal mindig a valós adatokat mutassa a felhasználóknak, és elkerüljük az adatok ütközését vagy a rendszer megakadását.

Bár a fejlesztés során voltak holtpontok, és sokat kellett tanulmányozni a különböző dokumentációkat, az elkészült webalkalmazás kárpótolt a befektetett munkáért. Ez a projekt rákényszerített minket a logikus problémamegoldásra, és nagyszerű gyakorlati alapot adott ahhoz, hogy a szoftverfejlesztést egy teljes, átfogó folyamatként lássuk át az adatbázisok tervezésétől egészen a kész grafikus felületig.

9.2. További fejlesztési lehetőségek

A jelenlegi webalkalmazás egy stabil, megbízható és jól működő alapot biztosít a foglalási folyamatok kezelésére, azonban egy szoftver sosincs teljesen készen. A projekt tervezése és megvalósítása során számos olyan további funkciót és irányt azonosítottunk, amellyel a rendszer a jövőben még kényelmesebbé, felhasználóbarátabbá és professzionálisabbá tehető.

Az egyik leginkább kézenfekvő továbblépési lehetőség egy automatizált értesítési rendszer bevezetése lenne. Ennek segítségével a felhasználók emailben vagy akár SMS-ben kaphatnának azonnali visszaigazolást a sikeres foglalásukról, illetve egy emlékeztetőt az időpontjuk előtti napon. Ez nemcsak az ügyfélélményt növelné, de a szalon számára is hasznos lenne, hiszen jelentősen csökkenthetné a feledékenységből adódó, lemondás nélküli távolmaradásokat. Ehhez az irányhoz szorosan kapcsolódhatna egy online fizetési modul integrálása is, amellyel az ügyfelek már a foglalás pillanatában, bankkártyával kifizethetnék a szolgáltatást vagy előleget hagyhatnának.

Kereskedelmi szempontból nagy potenciál rejlik az adminisztrációs felület továbbfejlesztésében is. A jövőben érdemes lenne egy vizuális, grafikonokkal ellátott statisztikai irányítópultot (dashboard) létrehozni a vezetőség számára. Ezen a felületen a fodrászok nyomon követhetnék a legnépszerűbb szolgáltatásokat, a legforgalmasabb napszakokat, valamint a havi bevételek alakulását. A vendégkör megtartása érdekében pedig egy beépített hűségprogram is kialakítható lenne, ahol a visszatérő profilok kedvezményeket kaphatnak.